Git Fundamentals

Module 3 · Session cheat sheet · Concise reference for the lab

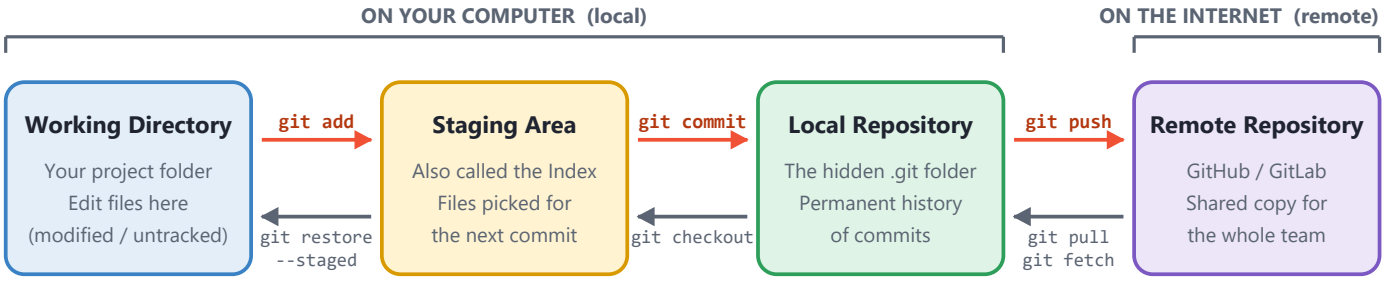
DAY 1

WHY VERSION CONTROL?

History	Undo	Collaborate
See who changed what, when and why	Roll back to any earlier version safely	Many people, one project, no overwriting

1 Git Architecture *the three areas a file travels through*

Git is a **distributed** version control system: every developer keeps the full project history on their own computer.



Working Directory	The files you see and edit. Git notices changes but is **not saving** them yet.
Staging Area	A holding zone for the **next commit**. You choose exactly which changes go in with `git add`.
Local Repository	The **.git** folder: permanent snapshots (commits) stored on your computer. `git commit` writes here.
Remote Repository	A copy hosted online (GitHub) for backup and sharing. `git push` sends commits there.

Parcel analogy: items on your table (working directory) → put into the box (staging) → seal and label the box (commit + message) → stored in the warehouse (repository).

Under the hood

- **Commit** = snapshot + message + author + unique ID (SHA) + link to its parent
- **Branch** = a lightweight, movable pointer to a commit
- **HEAD** = pointer to where you are right now

Git vs GitHub

- **Git** = the tool on your computer that tracks changes (works offline)
- **GitHub** = a website that hosts Git repositories for sharing and teamwork
- Alternatives to GitHub: GitLab, Bitbucket

2 Command Cheat Sheet *Git Lab commands are tagged LAB*

SETUP & START		
	`git config --global user.name "Name"`	Set who you are (also **user.email**); stamped on every commit
LAB	`git init`	Turn the current folder into a Git repository (creates **.git**)
	`git clone <url>`	Copy an existing remote repository to your computer
DAILY WORKFLOW		
LAB	`git status`	Show which files are modified, staged or untracked
LAB	`git add <file> | git add .`	Move changes to the **staging area** (one file / everything)
LAB	`git commit -m "message"`	Save the staged snapshot permanently in the **repository**
LAB	`git log --oneline --graph`	Show commit history, one line per commit, with branch graph
	`git diff`	See what changed but is not staged yet
BRANCHING & MERGING		
LAB	`git branch`	List branches (* marks the current one)
LAB	`git branch <name>`	Create a new branch
LAB	`git checkout <name>`	Switch to that branch (newer alias: **git switch**)
	`git checkout -b <name>`	Create a branch and switch to it in one step
	`git merge <name>`	Merge that branch into the branch you are currently on
	`git branch -d <name>`	Delete a branch after it has been merged
	`git merge --abort`	Cancel a merge that hit conflicts (back to before the merge)
IGNORE FILES		
	`git check-ignore -v <file>`	Show which .gitignore rule is ignoring a file
	`git rm --cached <file>`	Stop tracking a file but keep it on disk (then commit)
REMOTE (GITHUB)		
	`git remote add origin <url>`	Link your local repository to a GitHub repository
	`git push -u origin main`	Upload your commits to GitHub
	`git pull`	Download the latest commits and merge them in
UNDO		
	`git restore --staged <file>`	Unstage a file (changes stay in your working directory)
	`git restore <file>`	Discard unsaved edits in a file – **cannot be undone**

Typical flow: `git init` → `git add .` → `git commit -m "msg"` → `git branch` / `git checkout` → `git merge`

<angle brackets> = replace with your own value. The default branch is called **main** (**master** in older Git versions).

3 Creating a .gitignore *tell Git which files to never track*

Use it for secrets (**.env**), dependencies (**node_modules/**), build output, logs and OS / editor junk (**Thumbs.db**, **.DS_Store**).

1	**Create the file**	Name it exactly **.gitignore** (leading dot) in the project root, next to the **.git** folder. VS Code: New File, or `touch .gitignore` in Git Bash.
2	**Write patterns**	One pattern per line (see the table below). Example: **node_modules/**, **.env**, ***.log**.
3	`git status`	Ignored files disappear from the untracked list.
4	`git add .gitignore` `git commit -m "Add .gitignore"`	Commit the file so the whole team shares the same rules.
5	`git check-ignore -v <file>`	Optional: shows which rule is ignoring a file.

PATTERN CHEAT SHEET

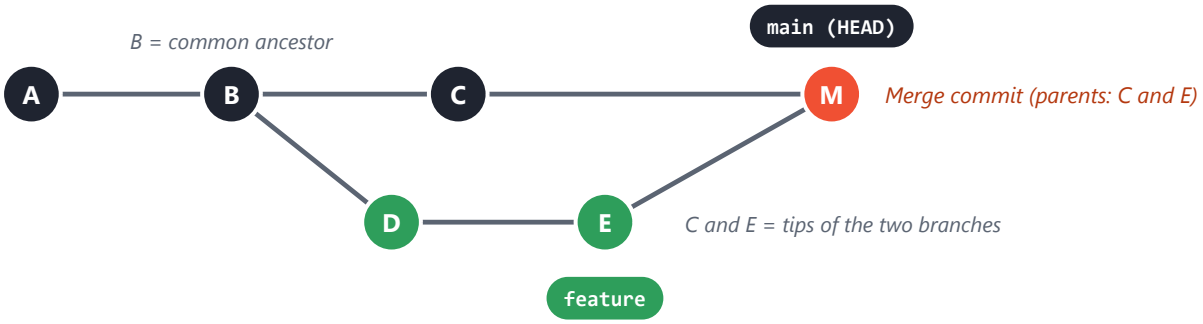
`# comment`	A comment line (ignored by Git)
`*.log`	Wildcard: every file ending in .log
`node_modules/`	A whole folder (note the trailing slash)
`.env`	One specific file, in any folder
`/notes.txt`	Only the file in the project root (leading slash)
`!important.log`	Exception: do **not** ignore this one

Already committed a file by mistake? .gitignore only affects untracked files. Run `git rm --cached <file>` (folder: `git rm -r --cached <folder>`) then commit – the file stays on your disk but Git stops tracking it.

4 Merging Principles *know these before you run git merge*

1 Merge INTO the receiving branch	Check out the branch that should **get** the changes (e.g. main), then run `git merge feature`. Only the current branch changes.
2 Start clean and up to date	Commit or stash your work first, and `git pull` so the target branch is current.
3 Three-way comparison	Git compares the **common ancestor** with the **tip of each branch** (diagram in section 5) and combines the changes.
4 Different lines: auto-merge	Changes in different files or different lines are combined automatically.
5 Same lines: conflict	If both branches changed the **same lines**, Git stops and asks you to decide.
6 History is never lost	The source branch is untouched; merging only adds commits to the target.
7 Two possible outcomes	**Fast-forward**: pointer just moves, no new commit. **Merge commit**: a new commit with two parents (`--no-ff` forces one).
8 Test after merging	Open or run the project before you push the result.

5 Merging Branches *step by step*



1	`git status`	Working tree must be clean. Commit your pending work first.
2	`git checkout main`	Switch to the branch that should **receive** the changes.
3	`git merge feature`	Bring the commits of **feature** into **main**.
4	`git log --oneline --graph`	Check the result. **Fast-forward** or **Merge made by...** = success. **CONFLICT** = sections 6 and 7.
5	`git branch -d feature`	Optional: delete the branch once it is merged.

6 Create a Merge Conflict on Purpose *classroom demo*

Goal: change the **same line** differently on two branches, then merge them. Use Git Bash or the VS Code terminal, and edit files in an editor – not with **echo >** in Windows PowerShell (it saves UTF-16, so Git treats the file as binary).

1	`mkdir conflict-demo`
	`cd conflict-demo`
	`git init`
		Create a fresh demo repository.

2	**Create index.html, save:**
	`<h1>My Portfolio</h1>`
	`git add .`
	`git commit -m "Base heading"`
	`git branch -M main`
		The **base commit**: the common ancestor of both branches. The last command names the branch **main** (older Git defaults to **master**).

3	`git checkout -b feature`	Create and switch to the **feature** branch.

4	**Edit line 1, save:**
	`<h1>Hi, I'm a CS student!</h1>`
	`git commit -am "Heading on feature"`
		Change made on **feature**. The **-a** flag stages modified tracked files.

5	`git checkout main`	Back on **main**. Re-open index.html: the old heading is back, so each branch holds its own version.

6	**Edit the same line 1, save:**
	`<h1>Welcome to my portfolio</h1>`
	`git commit -am "Heading on main"`
		Same line, different change on **main**.

7	`git log --oneline --graph --all`	Optional: show the class that the two branches have diverged.
8	`git merge feature`	**CONFLICT (content): Merge conflict in index.html**. Automatic merge failed; fix conflicts and then commit the result. Resolve it in section 7.

7 Resolving a Merge Conflict *five ways, pick one*

After the demo merge, Git has marked the clash inside **index.html**. Everything between `<<<<<<<` and `>>>>>>>` is a conflict zone:

```
<<<<<<< HEAD                          ← start of your version (current branch)
<h1>Welcome to my portfolio</h1>      ← your branch (main)
=======                               ← divider
<h1>Hi, I'm a CS student!</h1>        ← incoming branch (feature)
>>>>>>> feature                       ← end of incoming version
```

WAY A · EDIT BY HAND (works everywhere)

1	`git status`	Files under **Unmerged paths** are the ones to fix.
2	**Open the file**	Look at both versions and decide: keep yours, keep theirs, or **combine both**.
3	**Edit and clean up**	Delete the 3 marker lines (`<<<<<<<`, `=======`, `>>>>>>>`), keep only the final code, save.
4	`git add index.html`	Tells Git this file is **resolved** (stages it).
5	`git commit -m "Merge feature"`	Completes the merge.
6	`git log --oneline --graph`	Verify: the new merge commit has two parents.

OTHER WAYS TO RESOLVE

Way	How	Best when
B · VS Code buttons	Click **Accept Current** (main), **Accept Incoming** (feature) or **Accept Both** above the markers. Save, then `git add` and `git commit`.	Beginners, quick visual fix
C · Keep one side	`git checkout --ours index.html` keeps main's version; `git checkout --theirs index.html` takes feature's. Then `git add` and `git commit`.	You already know which version wins
D · Merge tool	`git mergetool` opens your configured tool (VS Code, Meld…) with a 3-pane view.	Big files, many conflicts
E · Abort	`git merge --abort` cancels the merge and returns to the state before it.	Not ready to decide yet

During a merge, **ours** = the branch you are on (main), **theirs** = the branch being merged in (feature).

Avoid conflicts: commit small and often, `git pull` before you push, and don't edit the same lines on two branches.

Remember: a conflict is normal, not an error. Git simply needs a human to choose.